

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

*I **Dhanush Kumar. S (192421096)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability.

Recommend a secure password hashing strategy and justify your choice.

1. Introduction

Password hashing is an important security mechanism used to protect user passwords in web applications. Instead of storing passwords directly, applications should store a **secure password hash**. However, the security of the stored password depends on both the **hashing algorithm** and the use of a **unique salt**.

In this case study, a web application stores passwords using **unsalted SHA-1 hashes**. After a database breach, attackers are able to recover many user passwords quickly using precomputed tables.

2. Problem Statement

A web application stores user passwords as:

SHA-1(password)

without adding a unique random salt.

When attackers obtain the password database, they can compare the stolen SHA-1 hashes against precomputed hash tables and efficiently recover passwords, particularly weak or commonly used ones.

The application therefore has a **weak password-storage mechanism** that exposes users to password cracking and credential reuse attacks.

3. How Unsalted Hashing Caused the Vulnerability

A **salt** is a random value generated for each password and stored alongside the resulting password hash.

Without a salt, the same password always produces the same hash.

For example:

Password: password123

SHA-1:

482c811da5d5b4bc6d497ffa98491e38...

If 1,000 users use the same password, all of them will have the same SHA-1 hash.

This creates several problems:

Attackers can use **precomputed rainbow tables** to find matching hashes.

Identical passwords produce identical hashes, revealing password reuse.

One precomputed lookup can potentially help crack passwords belonging to many users. Attackers can perform large-scale offline cracking after stealing the database.

With a unique salt:

Password + Random Salt → Password Hash

the same password produces different hashes for different users.

4. Why SHA-1 Is Inappropriate for Password Storage

SHA-1 is a **fast general-purpose cryptographic hash function**. That speed is useful for applications such as data integrity, but it is undesirable for password hashing.

Password hashing should deliberately be **slow and computationally expensive** so that attackers cannot test millions or billions of guesses efficiently.

SHA-1 also has known cryptographic weaknesses, including practical collision attacks. Although collision resistance is not the main reason SHA-1 is unsuitable for password storage, its age and design make it an inappropriate choice for modern password protection.

The major problem in this case is:

Fast SHA-1 + No Salt

↓

Efficient Offline Password Cracking

↓

Multiple Passwords Compromised

5. Attack Scenario

Suppose the database contains:

Username Stored Password Hash

User1	482c...
User2	7c222...
User3	482c...
User4	5baa...

User1 and User3 have the same hash.

An attacker can:

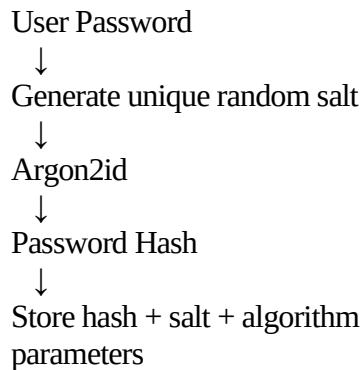
1. Steal the password database.
2. Identify that User1 and User3 use the same password.
3. Use a dictionary or precomputed table containing SHA-1 hashes.
4. Compare generated hashes with the stolen hashes.
5. Recover commonly used passwords.
6. Try the recovered username/password combinations on other websites.

This can lead to **credential stuffing**, especially because users frequently reuse passwords across multiple services.

6. Recommended Secure Strategy

The application should replace SHA-1 with a **dedicated password-hashing algorithm**, preferably **Argon2id**.

A secure architecture is:



Recommended choice: Argon2id

Argon2id is designed specifically for password hashing and provides resistance against large-scale offline attacks. It allows the application to configure:

Memory
usage
Time/cost
Parallelism

This makes password cracking significantly more expensive for attackers while remaining manageable for the legitimate application.

7. Alternative Secure Algorithms

Algorithm	Recommendation	Reason
Argon2id	★ Best choice	Modern, memory-hard password hashing
bcrypt	✓ Good	Widely supported and deliberately slow
scrypt	✓ Good	Memory-hard and designed for password hashing
PBKDF2-HMAC-SHA-256	✓ Acceptable	Useful where standards/FIPS compatibility is important

SHA-256/SHA-1 ❌ Not recommended alone Too fast for password storage

MD5 ❌ Not recommended Cryptographically broken and too fast

8. Salted Password Example

Instead of:

SHA-1("password123")

the application should use a unique random salt:

Password: password123

Salt: A8F3...random value

Argon2id(password123, salt)

↓

Stored password hash

For another user:

Password: password123

Different Salt: B91C...random value

Argon2id(password123, different salt)

↓

Different stored hash

Therefore, even when two users have the same password, their stored hashes will be different.

9. Secure Password Storage Design

The database can store something conceptually like:

User	Password Hash
------	---------------

Alice	Argon2id encoded hash containing salt + parameters
-------	--

Bob	Argon2id encoded hash containing different salt + parameters
-----	--

The salt does **not** need to be secret. It is normally stored with the password hash.

During login:

Entered Password

↓

Retrieve stored Argon2id parameters + salt

↓

Hash entered password

↓

Compare with stored hash



Match → Login successful

No Match → Login rejected

The application should use a well-maintained password-hashing library rather than implementing Argon2id itself.

10. Additional Security Measures

The application should also implement:

Unique random salt for every password

Appropriate Argon2id cost parameters

Secure password-reset mechanisms

Rate limiting for login attempts

Multi-factor authentication (MFA)

Protection against credential stuffing

HTTPS for password transmission

Secure session management

Regular security testing

Password breach detection where appropriate

The application should also **rehash passwords when users successfully log in** if the stored hash uses outdated parameters.

11. Comparison

Feature	Existing System	Recommended System
Algorithm	SHA-1	Argon2id
Salt	None	Unique random salt
Hash speed	Very fast	Deliberately expensive
Rainbow-table resistance	Poor	Strong
Offline cracking resistance	Poor	Much stronger
Password reuse visibility	Easy to detect	Hidden by unique salts
Modern suitability	✗ Insecure	✓ Recommended

12. Conclusion

The vulnerability occurred because the application combined **unsalted password storage with a fast, outdated hash function**. Unsalted SHA-1 allows attackers to reuse precomputed tables and efficiently crack common passwords after obtaining the database.

The recommended solution is to migrate to **Argon2id with a unique random salt for every password** and appropriately configured computational and memory costs. This makes offline password cracking substantially more expensive and provides a modern foundation for secure password storage.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

1. Introduction

Digital signatures are widely used in financial systems to authenticate users and approve transactions. A digital signature is generated using the user's **private key** and verified using the corresponding **public key**.

In this case, an attacker gains access to a user's private key and uses it to sign unauthorized financial transactions. Since the signatures are mathematically valid, the system may initially treat these transactions as legitimate.

2. Problem Statement

A financial institution uses digital signatures to authorize transactions. Each authorized user has a private key used to digitally sign transactions.

An attacker obtains the private key of a legitimate user and signs fraudulent transactions.

The system verifies the signatures successfully because the attacker possesses the correct private key. This creates serious problems involving:

- Authentication
- Authorization
- Transaction integrity
- Non-repudiation
- Financial loss

3. How the Attack Works

The normal transaction process is:

```
graph TD; A[User creates transaction] --> B[Transaction is hashed]; B --> C[Hash is signed using user's private key]; C --> D[Digital signature generated]; D --> E[Bank verifies signature using public key]; E --> F[Transaction approved];
```

After private-key compromise:

```
graph TD; A[Attacker obtains private key] --> B[Creates fraudulent transaction];
```

↓
Signs transaction using stolen private key

↓
Bank verifies signature successfully

↓
Fraudulent transaction may be approved

The important issue is that **cryptographic verification proves possession of the private key, not necessarily the physical identity of the person currently using it.**

4. Impact on Authentication

Digital signatures provide strong cryptographic authentication when the private key is under the legitimate user's control.

However, once the private key is stolen, the authentication mechanism is compromised.

The attacker can create signatures that appear to originate from the legitimate user.

For example:

Legitimate User → Private Key → Valid Signature

Attacker → Stolen Key → Valid Signature

The verification system sees both as valid signatures.

Therefore:

Compromise of the private key destroys the trust placed in signature-based authentication for that key.

The attacker may impersonate the legitimate user until the key is revoked or replaced.

5. Impact on Non-Repudiation

Digital signatures are intended to provide **non-repudiation**, meaning the signer should not easily be able to deny having authorized a transaction.

However, private-key compromise creates a major complication.

Suppose a fraudulent transaction contains a valid signature from User A.

The institution can technically verify:

Signature → User A's Public Key → Valid

But User A can legitimately argue:

"I did not create this transaction. My private key was compromised."

Therefore, the signature alone cannot reliably establish that the legitimate user personally authorized the transaction after the private key was compromised.

Effect:

The cryptographic signature remains mathematically valid.

The identity associated with the key appears valid.

But the evidence that the legitimate person actually authorized the transaction is weakened.

Investigation and legal resolution become more difficult.

6. Security Impact

Security Property Impact of Private-Key Compromise

Authentication ✗ Attacker can impersonate user

Authorization ✗ Unauthorized transactions may be approved Integrity

△ Fraudulent transactions can have valid signatures

Non-repudiation △ Evidence becomes disputed

Confidentiality Depends on how the key is used

Financial security ✗ Potential financial loss

7. Mitigation Strategies

A. Hardware Security Modules (HSMs)

Private keys should preferably be generated and stored inside a **Hardware Security Module (HSM)**.

An HSM prevents the private key from being directly exposed to application software or users.

Transaction



Application



HSM



Private Key signs transaction



Signature returned

The private key remains inside the HSM.

B. Hardware Security Tokens

For individual high-value transactions, private keys can be stored in secure hardware tokens or smart cards.

The private key cannot simply be copied like a normal file.

This significantly reduces the risk of key theft through malware or unauthorized file access.

C. Multi-Factor Authentication

Digital signatures should not be the only security mechanism for sensitive financial operations.

The institution should implement MFA such as

 Password
 +
 Hardware token / authenticator
 +
 Digital signature

For high-value transactions, additional verification can be required.

D. Transaction Confirmation

The system should require the user to explicitly confirm important transaction details before signing.

For example:

Transfer Amount: ₹5,00,000
Beneficiary: ABC Ltd.
Account: XXXX1234

Confirm transaction?

 ↓
 User approval
 ↓
 Digital Signature

This reduces the possibility of malware silently modifying transaction details.

E. Key Rotation

Private keys should have a defined lifetime.

Organizations should periodically:

1. Generate a new key pair.
2. Register the new public key.
3. Safely retire the old key.
4. Update certificates and access policies.

Key rotation limits the damage caused by long-term key exposure.

F. Key Revocation

If a private key is suspected to be compromised, it must be **revoked immediately**.

The institution can use mechanisms such as:

Certificate Revocation Lists (CRLs)
Online Certificate Status Protocol (OCSP)
Certificate/key status databases

The compromised key should no longer be accepted for new transactions

G. Real-Time Transaction Monitoring

Financial institutions should monitor unusual transactions.

For example:

Normal transaction:

₹10,000 → usual beneficiary

Suspicious transaction:

₹20,00,000 → new beneficiary

↓

Risk detection

↓

Additional verification

The system can flag transactions based on:

Unusual transaction amount
New beneficiary
Unusual location/device
Abnormal transaction frequency
Sudden changes in user behavior

H. Role-Based Access Control

Users should only receive the permissions necessary for their responsibilities.

For example:

Employee

↓

Can create transaction

Manager

↓

Can approve transaction

Finance Administrator

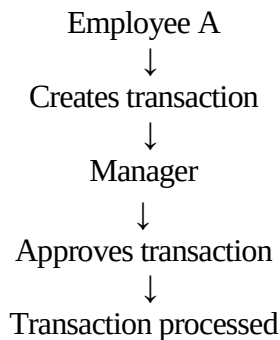
↓

Can approve high-value transaction

This follows the **principle of least privilege**.

I. Dual Authorization

High-value financial transactions should require approval from more than one authorized person.



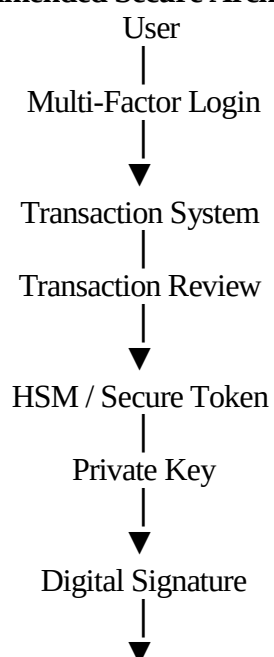
Even if one private key is compromised, the attacker cannot complete the transaction without the second authorization.

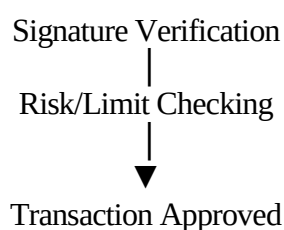
8. Incident Response after Key Compromise

If the institution discovers that a private key has been stolen, it should immediately:

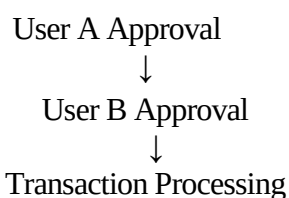
1. **Disable the compromised key.**
2. Revoke the associated certificate.
3. Stop or review transactions signed with the compromised key.
4. Generate a new private/public key pair.
5. Issue a new certificate.
6. Investigate how the private key was compromised.
7. Check system logs and transaction records.
8. Notify affected users and relevant authorities where required.
9. Strengthen access controls.
10. Monitor the new key for suspicious activity.

9. Recommended Secure Architecture





For high-value transactions:



10. Before and After Comparison

Feature	Existing System	Secure System
Private-key storage	Potentially exposed	HSM/Secure token
Authentication	Digital signature alone	MFA + digital signature
Transaction approval	Single authorization	Risk-based/dual authorization
Key management	Weak	Rotation + revocation
Monitoring	Limited	Real-time monitoring
High-value transactions	Same process	Additional verification
Compromised key response	Delayed	Immediate revocation
Non-repudiation	Weakened after compromise	Stronger with additional evidence

11. Conclusion

The compromise of a private key is a **critical security incident** in a financial system. Because anyone possessing the private key can generate mathematically valid signatures, an attacker can impersonate the legitimate user and potentially authorize fraudulent transactions.

The compromise also weakens **non-repudiation**, because the legitimate user can claim that the private key was stolen and the transaction was not personally authorized.

The institution should therefore use **HSMs or secure hardware tokens, MFA, key rotation, immediate revocation, transaction monitoring, dual authorization, and strong incident-response procedures.**

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

1. Introduction

Kerberos is a network authentication protocol that uses **tickets and time-sensitive authentication**. In a distributed system, the client, Kerberos Authentication Server (AS), Ticket Granting Server (TGS), and application servers must have closely synchronized clocks.

Kerberos uses timestamps to prevent **replay attacks**. Therefore, incorrect system time can cause legitimate authentication requests to be rejected.

2. Impact of Time Synchronization on Kerberos

Kerberos tickets and authenticators contain timestamps. The receiving server checks whether the timestamp is within an acceptable **clock-skew tolerance**.

For example:

Client time: **10:00:00**

Server time: **10:07:00**

Allowed clock difference: **5 minutes**

The 7-minute difference is greater than the allowed tolerance, so the server may reject the authentication request.

Major effects:

1. **Ticket rejection**

A valid Kerberos ticket may be rejected if its timestamp appears too old or too far in the future.

2. **Authentication failures**

Users may repeatedly receive login or "authentication failed" errors even when their passwords and credentials are correct.

3. **Service disruption**

Applications depending on Kerberos authentication may become unavailable.

4. **Replay protection problems**

Kerberos relies on timestamps to identify and reject previously captured authentication messages. Poor synchronization weakens this mechanism and can create security risks if clock-skew controls are relaxed excessively.

- 5.

Ticket validity issues

Tickets have defined lifetimes. Incorrect clocks can make a ticket appear expired before its

3. Causes of Time Synchronization Problems

Common causes include:

Different time zones or incorrect time-zone configuration

Servers not using a common NTP source

Network connectivity problems with time servers

- Incorrect system clock configuration
- Virtual machines experiencing clock drift
- Hardware clock problems
- NTP service being stopped or misconfigured

4. Solutions

A. Use NTP

Deploy **Network Time Protocol (NTP)** across all machines.

A reliable architecture can be:

Central/Trusted Time Source → Domain/Network Time Servers → Kerberos Clients & Servers

All Kerberos-related systems should synchronize their clocks with trusted time sources.

B. Configure Multiple Time Servers

Use multiple trusted NTP servers rather than depending on a single server. This provides redundancy if one time server becomes unavailable.

C. Monitor Clock Drift

Regularly monitor the difference between:

- Client and KDC
- KDC and application servers
- Different authentication servers

Generate alerts when clock drift approaches the configured tolerance.

D. Configure an Appropriate Clock-Skew Limit

Kerberos implementations provide a clock-skew tolerance. The value should be large enough to accommodate normal network and clock variations but **not unnecessarily large**.

Increasing the tolerance excessively can increase the window in which replayed authentication information might be accepted.

E. Synchronize Virtual Machines

For virtualized servers, enable appropriate guest/host time synchronization and ensure that the VM's NTP configuration does not conflict with the hypervisor's time-management mechanisms.

F. Secure the Time Infrastructure

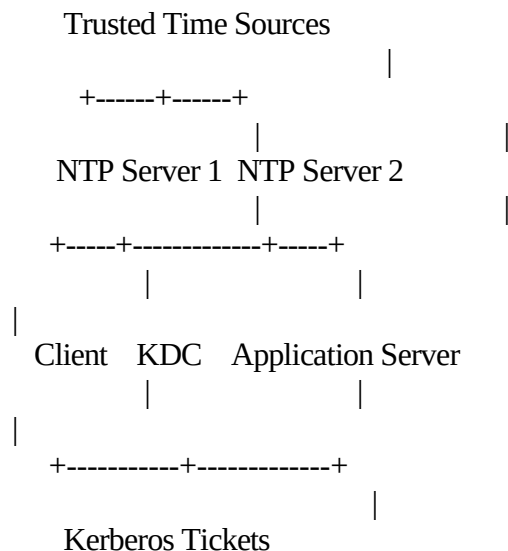
Time synchronization itself should be protected because an attacker who can manipulate system clocks could potentially interfere with authentication.

Use:

- Trusted NTP sources

- Network access controls
- Secure administration of time servers
- Monitoring and logging
- Authentication mechanisms for time synchronization where supported

5. Recommended Architecture



All components maintain synchronized clocks, allowing Kerberos timestamps and ticket lifetimes to be validated correctly.

6. Conclusion

Time synchronization is **critical to Kerberos authentication** because Kerberos uses timestamps to validate tickets and protect against replay attacks. Significant clock differences can cause legitimate authentication requests to fail and can disrupt distributed applications.

The best solution is to establish a **centralized and redundant NTP infrastructure**, continuously monitor clock drift, maintain a reasonable Kerberos clock-skew tolerance, and secure the time servers. This provides reliable authentication while preserving Kerberos's security properties.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

1. Introduction

A **Certificate Authority (CA)** is a trusted entity in a **Public Key Infrastructure (PKI)** that issues and signs digital certificates. X.509 certificates bind an identity, such as a domain name, to a public key.

If a CA is compromised and an attacker obtains the ability to issue fraudulent certificates, the attacker can create a certificate for a trusted domain even though they do not own that domain.

2. Impact of a CA Breach

A. Loss of Trust

Browsers, operating systems, and applications automatically trust certificates issued by trusted CAs. A compromised CA can therefore undermine the trust model of the entire PKI.

For example:

Legitimate:

User → www.example.com

↓

Valid Certificate

↓

Trusted CA

Attack:

User → Attacker

↓

Fake Certificate for www.example.com

↓

Compromised Trusted CA

The browser may incorrectly consider the attacker's certificate trustworthy.

B. Man-in-the-Middle (MITM) Attacks

An attacker can use the fraudulent certificate to impersonate a legitimate server.

The attacker could potentially:

- Intercept HTTPS traffic

- Decrypt information exchanged with the attacker-controlled endpoint

- Modify communication

- Steal credentials or session information

- Redirect users to malicious services

C. Loss of Confidentiality and Integrity

Secure communication depends on certificates to authenticate the server. A fraudulent certificate can allow an attacker to impersonate the server, compromising both:

Confidentiality – sensitive information may be exposed.

Integrity – communications may be modified without the user realizing it.

D. Large-Scale Impact

A CA breach can affect many organizations because a single trusted CA may issue certificates for thousands or millions of domains.

The compromise can therefore have a much wider impact than an attack against one individual website.

3. Detection Mechanisms

A. Certificate Transparency (CT)

Certificate Transparency logs publicly record certificates issued by CAs.

Organizations can monitor CT logs for unexpected certificates issued for their domains.

For example:

Expected:

example.com → Certificate A

Detected in CT:

example.com → Certificate A

example.com → Certificate B

example.com → Certificate C ← Suspicious

Unexpected certificates can indicate unauthorized issuance.

B. Certificate Monitoring

Organizations should continuously monitor certificates associated with their domains and identify:

Unknown certificates

Unexpected issuers

Unexpected public keys

Certificates with unusual validity periods

Certificates issued for unauthorized subdomains

C. OCSP

Online Certificate Status Protocol (OCSP) allows clients to check whether a certificate has been revoked.

If a fraudulent certificate is identified and revoked, OCSP can help clients determine that it should no longer be trusted.

D. CRLs

Certificate Revocation Lists (CRLs) contain certificates that have been revoked by the CA.

Systems can periodically download CRLs and reject certificates appearing on the list.

E. DNS-Based Monitoring

CAA (Certification Authority Authorization) DNS records can specify which CAs are authorized to issue certificates for a domain.

For example:

example.com
|
+--- CAA → Only CA-A authorized

If another CA attempts to issue a certificate, it should not be authorized to do so.

4. Mitigation Strategies

A. Revoke Fraudulent Certificates

Once a fraudulent certificate is discovered, the CA should immediately revoke it using mechanisms such as:

- OCSP
- CRLs

Affected organizations should also replace compromised certificates.

B. Remove or Restrict the Compromised CA

Browsers and operating systems can **remove or distrust the compromised CA's root certificate**.

This prevents newly issued fraudulent certificates from being accepted.

C. Secure CA Private Keys

CA private keys are extremely sensitive. They should be protected using:

- Hardware Security Modules (HSMs)
- Strong access controls
- Multi-person authorization
- Secure key storage
- Regular security audits

D. Use Short-Lived Certificates

Short certificate lifetimes reduce the amount of time a fraudulent certificate can remain useful.

For example:

Long-lived certificate
↓
Larger attack window

Short-lived certificate



Smaller attack window

E. Implement Certificate Pinning Carefully

Applications can sometimes restrict connections to expected certificates or public keys. This can reduce the effectiveness of fraudulent certificates.

However, traditional certificate pinning can create operational problems when certificates are legitimately renewed, so it should be used carefully.

F. Continuous CT Monitoring

Organizations should automatically monitor Certificate Transparency logs and immediately investigate unexpected certificates.

5. Incident Response

When a CA breach occurs, the following procedure should be followed:



6. Conclusion

A CA breach is a **critical PKI security incident** because it breaks the fundamental trust relationship between users and digital certificates. Attackers may use fraudulent X.509 certificates to impersonate trusted domains and perform MITM attacks.

The most effective defense is a combination of **Certificate Transparency monitoring, CAA records, certificate revocation, secure CA key management, short-lived certificates, strong access controls, and rapid incident response**. These mechanisms help detect unauthorized certificates and restore trust in the PKI environment.

5. Integrity Verification in Cloud Data Storage

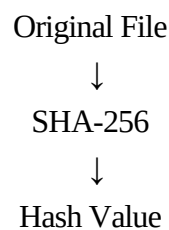
A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

1. Introduction

Hashing is commonly used in cloud storage to verify whether a file has been modified. A **cryptographic hash function** converts a file into a fixed-length value called a hash or digest.

For example:



If the file changes, its hash should also change. However, improper implementation can allow attackers to modify data without the system detecting the tampering.

2. How Incorrect Hashing Causes Integrity Failures

A. Using Weak Hash Algorithms

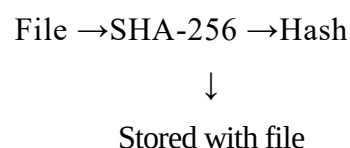
Using outdated algorithms such as **MD5** or **SHA-1** can create security weaknesses because practical collision attacks exist against them.

An attacker may construct different data that produces the same hash, potentially bypassing integrity checks.

Solution: Use modern algorithms such as **SHA-256** or **SHA-3**.

B. Storing the Hash Alongside the File

A common mistake is:



If an attacker can modify both the file and its stored hash, they can simply calculate a new hash for the modified file.

For example:

Original:

File A → Hash

A

After attack

Modified File → Hash B



Replace Hash A with B

The verification system may incorrectly conclude that the file is valid.

Solution: Store the integrity value in a location protected from unauthorized modification.

C. Using an Unauthenticated Hash

A normal hash provides integrity detection only if the expected hash itself is trusted.

A stronger mechanism is an **HMAC (Hash-based Message Authentication Code)**:

File + Secret Key



HMAC



Authentication Tag

An attacker who does not know the secret key cannot easily generate a valid HMAC for modified data.

D. No Secure Hash Comparison

Improper comparison of hash values can introduce implementation weaknesses.

The system should compare the calculated digest with the trusted expected digest using a secure comparison mechanism rather than unsafe string-processing logic.

E. Hash Calculated Only During Upload

If the cloud system calculates a hash only when a file is uploaded and never checks it again, later corruption or tampering may remain undetected.

Solution: Perform periodic or on-demand integrity verification.

F. No Versioning or Audit Logs

Without file versioning and audit logs, the system may detect that a file changed but be unable to determine:

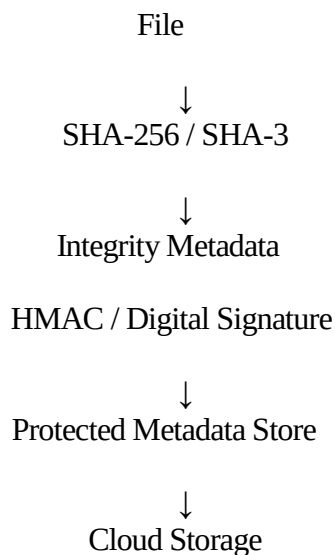
- When it changed
- Who changed it
- Which version was legitimate
- Whether the modification was authorized

Therefore, integrity verification should be combined with logging and version control

3. Recommended Robust Integrity Verification Mechanism

A strong solution is to use **SHA-256/SHA-3 combined with authenticated integrity protection, secure metadata storage, versioning, and periodic verification.**

A suitable architecture is:



4. Verification Process

Step 1: Upload

When a user uploads a file:

File → SHA-256 → Digest

The system generates an integrity value.

Step 2: Protect the Integrity Value

Instead of storing the digest where an attacker can modify it, protect it using:

**HMAC with a securely stored secret key, or
Digital signature using a protected private key**

The protected integrity information should be stored separately from the user-accessible file.

Step 3: Store the File

The encrypted/protected file and its integrity metadata are stored in cloud storage.

Step 4: Verification

When the file is downloaded or periodically checked:

Stored File



Calculate SHA-256

Compare with Trusted Integrity Value



↓ ↓
Valid Tampering/
 Corruption

Step 5: Alert and Recovery

If the calculated value does not match the trusted value:

- Mark the file as suspicious.
- Generate a security alert.
- Record the incident in audit logs.
- Prevent distribution of the corrupted file.
- Restore a known-good version using cloud versioning or backup.

5. Additional Security Measures

Mechanism	Purpose
SHA-256 / SHA-3	Strong cryptographic hashing
HMAC	Prevents unauthorized modification of integrity metadata
Digital signatures	Provides integrity and origin authentication
Encryption	Protects confidentiality

Access control	Prevents unauthorized modification
Versioning	Enables recovery of previous versions
Audit logs	Tracks file operations
Periodic verification	Detects delayed tampering
Secure key management	Protects HMAC/signing keys

6. Why This Mechanism Is Effective

The recommended approach is effective because **the attacker cannot simply modify the file and generate a new valid integrity value.**

With an ordinary hash:

Modified File → New Hash

An attacker who can modify the stored hash may bypass verification.

With HMAC:

Modified File + Secret Key
↓
New HMAC

Without access to the secret key, the attacker cannot generate the correct authentication tag.

Digital signatures provide a similar benefit while also providing **authentication and non-repudiation** when appropriately implemented

7. Conclusion

Incorrect hashing practices—such as using weak algorithms, storing hashes where attackers can modify them, failing to authenticate the hash, and not performing periodic verification—can result in undetected cloud data tampering.

A robust solution should use **SHA-256 or SHA-3 with HMAC or digital signatures, secure key management, protected integrity metadata, access control, versioning, audit logs, and regular integrity checks.** This ensures that unauthorized modifications are detected quickly and legitimate data can be reliably restored.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand